SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning,*

and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I _S.Jotheeswaran_(192424155)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.Jotheeswaran

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, − 1 each step, − 5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Integrated Experiments Solution

Experiment 1: Decision Tree Classification

- Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python and scikit-learn.

(a) Data Loading, Pre-processing, and Splitting

The Iris dataset is loaded using scikit-learn. Since it contains clean continuous features and no missing values, missing value handling is bypassed. Categorical targets are automatically encoded by scikit-learn. The data is split into 70% training and 30% testing sets.

Python Implementation:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target, name='Target')

# Display first 5 rows and data types
print("First 5 rows:")
print(X.head())
print("\nData Types:")
print(X.dtypes)

# Split dataset (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

(b) Building the Decision Tree Model

We use the Gini Index as the splitting criterion. The root node is identified based on the feature that provides the lowest Gini impurity (or highest information gain) for the entire dataset.

```
from sklearn.tree import DecisionTreeClassifier, export_text

# Build model using Gini Index
clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)
```

```
# Print tree structure
tree_rules = export_text(clf, feature_names=list(X.columns))
print("Decision Tree Structure:\n", tree_rules)
```

Root Node Justification: The tree visualization typically shows 'petal width (cm)' or 'petal length (cm)' as the root node. This is mathematically justified because this feature perfectly separates the Setosa species from the Versicolor and Virginica species, resulting in the most significant reduction in Gini impurity (maximum information gain) at the first split.

(c) Model Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import numpy as np

# Predict on test set
y_pred = clf.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Identify misclassified instances
misclassified_idx = np.where(y_test != y_pred)[0]
if len(misclassified_idx) > 0:
    print("\nMisclassified Instances:")
    for idx in misclassified_idx:
        actual = iris.target_names[y_test.iloc[idx]]
        predicted = iris.target_names[y_pred[idx]]
        features = X_test.iloc[idx].to_dict()
        print(f"Features: {features} | Actual: {actual} | Predicted: {predicted}")
else:
    print("\nNo misclassified instances found in this test split.")
```

Performance Comment: The Decision Tree usually achieves near 100% accuracy on the Iris dataset. If instances are misclassified, they typically occur at the boundary between Versicolor and Virginica, as their sepal and petal dimensions overlap slightly. To demonstrate misclassification, one might reduce the training size or change the random state.

Experiment 2: Reinforcement Learning – Q-Learning

(a) Environment Definition

We define a 4x4 grid.
- Actions: Up (0), Down (1), Left (2), Right (3)
- Rewards: +10 for Goal (3,3), -5 for Obstacles at (1,1) and (2,2), -1 for every other step.
- Hyperparameters: Alpha (learning rate) = 0.1, Gamma (discount factor) = 0.9, Epsilon (exploration rate) = 0.1.

```
import numpy as np
import random

# Environment parameters
grid_size = 4
num_states = grid_size * grid_size
num_actions = 4  # 0: Up, 1: Down, 2: Left, 3: Right
goal_state = 15  # (3, 3)
obstacles = [5, 10]  # (1, 1) and (2, 2)

# Hyperparameters
alpha = 0.1  # Learning rate
gamma = 0.9  # Discount factor
epsilon = 0.1  # Exploration rate

# Initialize Q-table with zeros
Q_table = np.zeros((num_states, num_actions))
```

(b) Q-Learning Algorithm (100 Episodes)

The Q-learning algorithm iterates through 100 episodes. We track the Q-values for two representative states, for example, State 0 (Start) and State 14 (next to Goal), at episodes 1, 50, and 100.

```
def step(state, action):
    row, col = divmod(state, grid_size)
    if action == 0 and row > 0: row -= 1
    elif action == 1 and row < grid_size - 1: row += 1
    elif action == 2 and col > 0: col -= 1
    elif action == 3 and col < grid_size - 1: col += 1

    next_state = row * grid_size + col

    if next_state == goal_state: return next_state, 10, True
    elif next_state in obstacles: return next_state, -5, False
```

```
    else: return next_state, -1, False

# Training Loop
rewards_per_episode = []
tracked_states = [0, 14]
q_evolution = {s: [] for s in tracked_states}

for episode in range(1, 101):
    state = 0
    total_reward = 0
    done = False

    while not done:
        if random.uniform(0, 1) < epsilon:
            action = random.choice([0, 1, 2, 3])
        else:
            action = np.argmax(Q_table[state])

        next_state, reward, done = step(state, action)
        total_reward += reward

        # Q-Learning Update Rule
        best_next_action = np.argmax(Q_table[next_state])
        td_target = reward + gamma * Q_table[next_state, best_next_action]
        td_error = td_target - Q_table[state, action]
        Q_table[state, action] += alpha * td_error

        state = next_state
        if total_reward < -50: break # Avoid infinite loops in early training

    rewards_per_episode.append(total_reward)

    if episode in [1, 50, 100]:
        for s in tracked_states:
            q_evolution[s].append(Q_table[s].copy())
```

(c) Learned Policy and Convergence

The optimal policy is extracted by taking the argmax of the Q-values for each state.

```
# Extract Policy
actions = ['Up', 'Down', 'Left', 'Right']
policy = []
for state in range(num_states):
```

```
        if state == goal_state:
            policy.append("GOAL")
        elif state in obstacles:
            policy.append("OBST")
        else:
            policy.append(actions[np.argmax(Q_table[state])])

    print("Learned Policy (4x4 Grid):")
    for i in range(0, num_states, grid_size):
        print(policy[i:i+grid_size])

    # Code to plot (using matplotlib)
    import matplotlib.pyplot as plt
    plt.plot(rewards_per_episode)
    plt.title('Cumulative Reward per Episode')
    plt.xlabel('Episode')
    plt.ylabel('Total Reward')
    plt.show()
```

Convergence Justification: As the episodes progress from 1 to 100, the cumulative reward stabilizes and plateaus, indicating that the agent has successfully learned an optimal path to the goal while avoiding obstacles. The Q-values for states near the goal converge to higher positive values, correctly propagating the +10 reward backwards through the grid.